

4- Five people P, Q, R, S and T are standing in a queue. R is standing between P and T. P is just behind Q and Q is second in the queue. Who is second last in the queue?

- a- T b- S c- R d- P

5- Which of the following data structures can't access randomly?

- a- Arrays b- Linked List c- A+B d- None of them

6- Which among the following is the proper syntax for the template class?

- a) `template <typename T1, typename T2>;` b) `Template <typename T1, typename T2>;`
c) `template <typename T> T named(T x, T y){ }`
d) `Template <typename T1, typename T2> T1 named(T1 x, T2 y){ }`

7- A template class can have _____

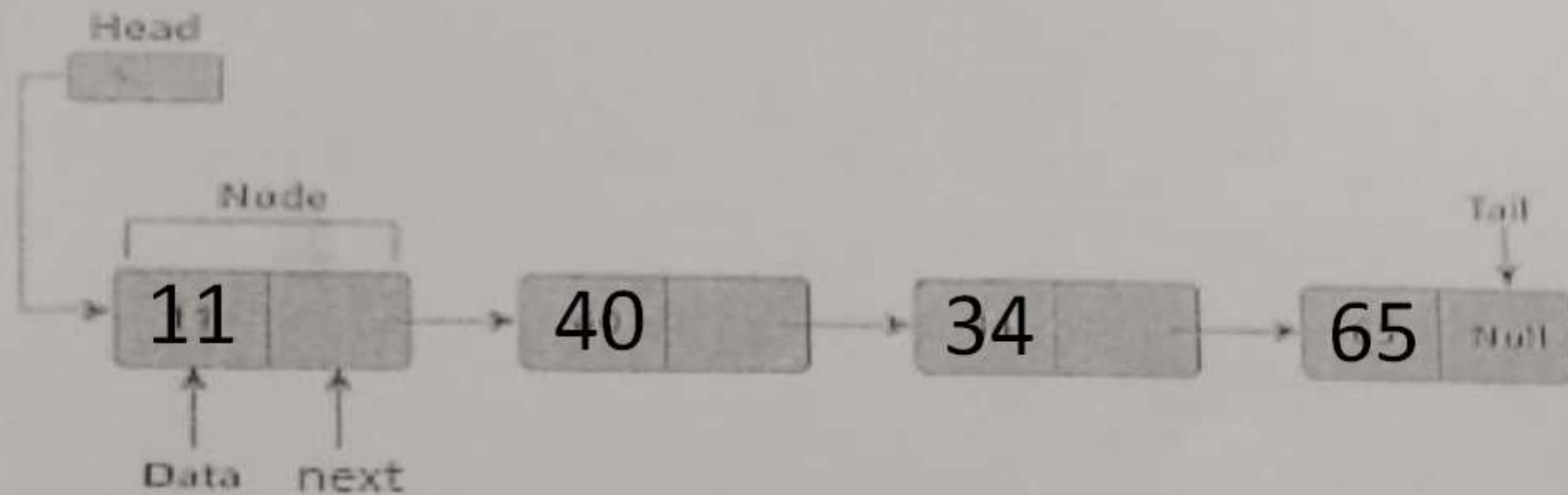
- a) More than one generic data type b) Only one generic data type
c) At most two data types d) Only generic type of integers and not characters

7) Use the following LinkList to determine the value of following instructions

```
node <T> *q;
```

```
q= head-> next->next;
```

```
cout<<q->next->data;
```



☒ A. 65

B. 40

C. 34

D. 11

- 1) The value of REAR is increased by 1 when
- A. An element is deleted in a queue
 - B. An element is traversed in a queue
 - ☒ C. An element is added to a queue
 - D. An element is merged in a queue
- 2) The insertion operation in the stack is called
- A. insert
 - ☒ B. push
 - C. pop
 - D. enqueue
- 3) Link fields hold pointers to the element in the linked representation of the stack.
- ☒ A. Neighboring
 - B. Last
 - C. First
 - D. Middle
- 4) Which of the following is **TRUE** about destructors?
- A. Its name is the class name preceded by a ~ (tilde)
 - B. It is a special member function that works just opposite to the constructor
 - C. Destructors are used to destroy (or delete) the object
 - ☒ D. All of the above
- 5) Which of the following is **FALSE** about Circular Queue?
- A. It helps in refilling all the free spaces by resetting REAR to zero when the REAR reaches the end of the queue ✓
 - ☒ B. It follows the Last in First out LIFO (LIFO) principle
 - C. It connects the last position of the queue to the first position of the queue ✓
 - D. None of the above
- 6) A destructor is automatically called when
- A. The program finished execution.
 - B. Calling the delete operator.
 - C. A scope (the { } parenthesis) containing local variable ends.
 - ☒ D. All of the above

Problem # 5: Write a C++ program to find the average of all elements stored in the queue? [5pts]

```
T findAvg()
{
    T mySum = 0;
    int i;
    float c=0.0;
    for (i = front + 1 ; i <=rear ; i++ )
        mySum += arr[i];
    c = i;
    return mySum/c;
}
```

// Programmed By : Laith Amro

- 13) Which of the following is useful in the queue structure
- ☒ A. Controlling printer jobs
 - B. Control first-in-last-out operations in the processor ✕
 - C. Arrange data in the file
 - D. None of the above



7) Given the class definition in the table below, which data member you cannot call by an object and the dot operator from outside the class

```
class MyClass{  
    public: int x,y;  
    private:int a;  
};
```

A. x

B. y

☒ C. a

D. None of the above

8) Class and function templates allow the programmer to write one code for all data types and help in generic programming.

☒ A) True

B) False

9) Which of the following is TRUE about linear Queue?

☒ A) It is also called Last-In-Last-Out list (LILO)

B. It becomes full when $\text{Front} = \text{size} - 1$

C. It becomes empty only when $\text{Rear} = -1$ and $\text{Front} = -1$.

D. None of the above

Problem # 4: Write a C++ program to find the maximum element stored in the stack. [5pts]

```
template <class T>
T stack<T>::myMax()
{
    if(tos==-1)
        cout<<"List Is Empty\n";
    else
    {
        T myMaxVar = array[tos];
        for(int i = tos ; i != -1 ; i-- )
            if(array[i] > myMaxVar)
                myMaxVar = array[i];
        return myMaxVar;
    }
}
```

//Programmed By : Laith Amro

ليث عمرو


```

#include <iostream>
using namespace std;
template <class T, class U, class V = char>
class ClassTemplate {
private:
    T var1;
    U var2;
    V var3;
public:
    ClassTemplate(T v1, U v2, V v3)
    {
        var1=v1;
        var2=v2;
        var3=v3;
    }
    void printVar() {
        cout << "var1 = " << var1 << endl;
        cout << "var2 = " << var2 << endl;
        cout << "var3 = " << var3 << endl;
    }
};

int main() {

    ClassTemplate<int, double> obj1(20, 19.8, 'd');
    cout << "obj1 values: " << endl;
    obj1.printVar();
    ClassTemplate<double, char, bool>
    obj2(12.8, 'c', true); cout << "\nobj2 values: " << endl;
    obj2.printVar();
    return 0;
}

```

Output

```

obj1 values:
var1 = 20
var2 = 19.8
var3 = d

```

```

obj2 values:
var1 = 12.8
var2 = c
var3 = 1

```


14) Undo/Redo operations in Excel or Word processing programs are a real example of linear queue data structure.

☐ A. True

☒ B. False

15) Linked lists allow random memory access.

☐ A. True

☒ B. False

- 11) In the linear queue data structure, the value of **Front** is increased by 1 when
- ☒ A. An element is deleted in a queue
 - ☐ B. An element is traversed in a queue
 - ☐ C. An element is added to a queue
 - ☐ D. An element is merged in a queue

Problem # 3: Write a C++ program to find the sum of all even numbers stored in the unordered link list. [5pts]

```
template <class T>
T unorderedlinklist<T>::SumAllEven()
{
    if (head == NULL)
    {
        cout << "List is empty!" << endl;
    }
    else
    {
        node *r;
        r = head;
        T SumAllEven=0;
        while(r!= NULL)
        {
            if(r->data%2 == 0)
                SumAllEven+=r->data;
            r = r->link;
        }
        return SumAllEven;
    }
}
```

// Programmed By : Laith Amro

ليث عمرو

2) The deletion operation in the stack is called

A. insert

B. push

☒ C. pop

D. enqueue

6) Which of the following is **incorrect** about the stack?

A. If $\text{top} = -1$, then the stack is empty

☒ B. If $\text{top} = -1$, then the stack is not empty

C. $\text{pop}()$ operation will remove (access) an element from the stack

D. None of the above

1- fill the blanks for the following function which return the count of the elements of an orderedlinklist that not equals x; (7 pts)

```
int fun(node* head, int x)
```

```
{  
  node* q;  int c = 0;
```

```
  for (q=head; -----; -----)  
    if (-----)
```

```
      c++;
```

```
  return c;}
```

q!=NULL

q=q->link

q->data != x

2) fill the blank for the following function which add 2 to every element in the queue

```
void queue<T>::fun() {  
    for(-----; i++;)  
        -----  
}
```

int i = front + 1

i <= rear

arr[i] = arr[i] + 2

Q4) Correct the errors in the following codes if found (8 pts)

```
#include <iostream>
using namespace std;
template <class T,class U>

class list
{
    public:
        U Fun2(T,U);
};

template <class T>
Fun2(T num1,U num2)
{
    return (num1+num2);
}

void main(){
    linklist<int> L;
    cout<<Fun2(5,7.5)
}
```

```
using namespace std;
template <class T,class U>

class list
{
    public:
        U Fun2(T,U);
};

template <class T,class U> //Edited
U list<T,U>::Fun2(T num1,U num2) // Edited
{
    return (num1+num2);
}

int main(){ // Edited
    list<int,float> L; // Edited
    cout<<L.Fun2(5,7.5); // Edited
    return 0; // Added
}
```

1- After execution the following code, the tos value are

```
stack <char> s;  
s.push(8);  
s.push(5);  
s.pop();  
s.push(4);  
s.push(3);  
s.pop();  
s.push(9);
```

1- tos=2

2- tos=0 3- tos=1 4- tos=3

2- After execution the following code, the context of queue is ----

```
queue<char> q;  
q.enqueue(5);  
q.enqueue(4);  
q.dequeue();  
q.enqueue(9);  
q.dequeue();
```

```
q.enqueue(1);
```

1- 9,1

2- 4,9,1

3- 5,4,9,1

4- 5,1

3- if (rear=front) This instruction checks if the

a. linklist is full

b. linklist is empty

c. Queue is full

d. Queue is empty

ليث عمرو

Problem # 3: Write a program to explain class template by creating a template T for a class named **Minimum_value** having three data members (**num1**, **num2**, and **num3**) of type T which are inputted by a constructor and a member function **get-minimum()** return the minimum of the three numbers to main. Note: the value of T depends upon the data type specified during object creation. [6pts]

```
#include <iostream>
using namespace std;

// Template class declaration
template <class T>
class Minimum_value {
private:
    T num1, num2, num3;
public:
    Minimum_value(T a, T b, T c)
    {
        num1 = a;
        num2 = b;
        num3 = c;
    }
    T get_minimum() {
        if (num1 < num2 && num1 < num3) {
            return num1;
        } else if (num2 < num3) {
            return num2;
        } else {
            return num3;
        }
    }
};
```

// Programmed By : Laith Amro

ليث عمرو